Felix Céard-Falkenberg
7174020

1 Question 5

(a) **Prove that C is a vertex cover of G.**

C is the set of all nodes with at least one children in a depth-first search tree of G.

First, if a node $v \in V$ has degree 1, i.e. there exists only one edge $(u, v) \in A$, then every depth-first search tree will contain v in the leaf and u in the first layer above the leaf. Therefore, u will be added to C.

Because the depth-first search tree will have at least one leaf for each node $v \in V$ with degree $d(v) > 1$, we know that by definition, each of these nodes will be added to C. Therefore, C is the set of all nodes with degree at least 2, with the exception of the graph containing only two nodes, in which case C contains one of the two nodes.

Because every node has an edge to a node in C, we know that C is a vertex cover of G and therefore the algorithm is correct.

(b) **Show that there is a matching in T with at least $r/2$ edges.**

Let us prove that there exists a matching in T with at least $r/2$ edges.

Because T is a tree, we can alternate between edges that are included and excluded from the matching. Because we have r non-leaf nodes in T, we can create a matching with at least $\lfloor r/2 \rfloor$ edges. However, if r is odd, we can add one more edge to the matching, namely the edge between the leaf and its parent for each branch in the tree, and therefore create a matching with at least $r/2$ edges. If r is even, then we already have a matching with at least $r/2$ edges.

(c) **Show that G has a matching of size at least $|C|/2$.**

Let us prove that G has a matching of size at least $|C|/2$.

Because C is a valid vertex cover of G, we can create a matching of size at least $|C|/2$ by selecting one edge from each node in C. Because each edge is incident to at least one node in C, we know that the matching is valid.

By adding the edges to the leafs — if $|C|$ is odd, otherwise $|C|/2$ is already guaranteed — we can guarantee that the matching is of size at least $|C|/2$.

(d) **Prove that the algorithm outputs a vertex cover whose size is at most two times the size of a minimum vertex cover.**

This is not true I think. Consider the graph in Figure 1. Either you do not include all nodes in the leafs, however, the only node not in the leaf is the root node, which, if $C = \{D\}$, is not a valid vertex cover. Otherwise, in this case, the algorithm would return $C = V$ although the minimal vertex cover is $C = \{E\}$. Because $|C| = 1 < 2 \cdot |C| < |V|$, I think this is not true, or the definition is ambiguous.

Felix Céard-Falkenberg
7174020

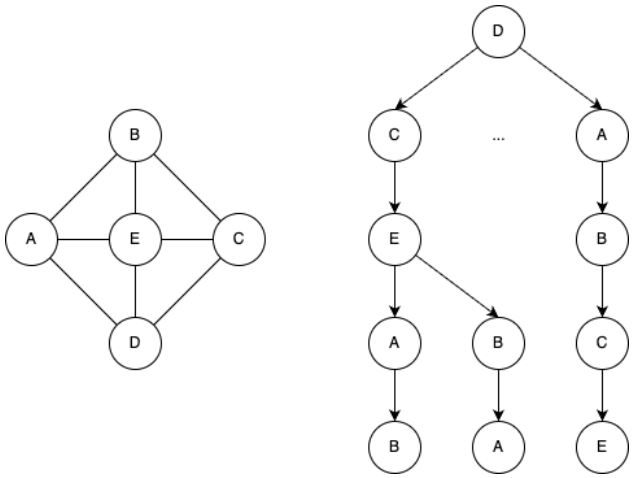

Figure 1: Left: An example graph. Right: an excerpt of the depth-first search tree.

(e) **The algorithm above was stated for connected graphs. What would you do for graphs that are not connected?**

Start the search for each node not already found. During the DFS, you keep a set of all checked nodes K and if $|K| < |V|$, you start a new DFS from a node $v \in V \setminus K$.